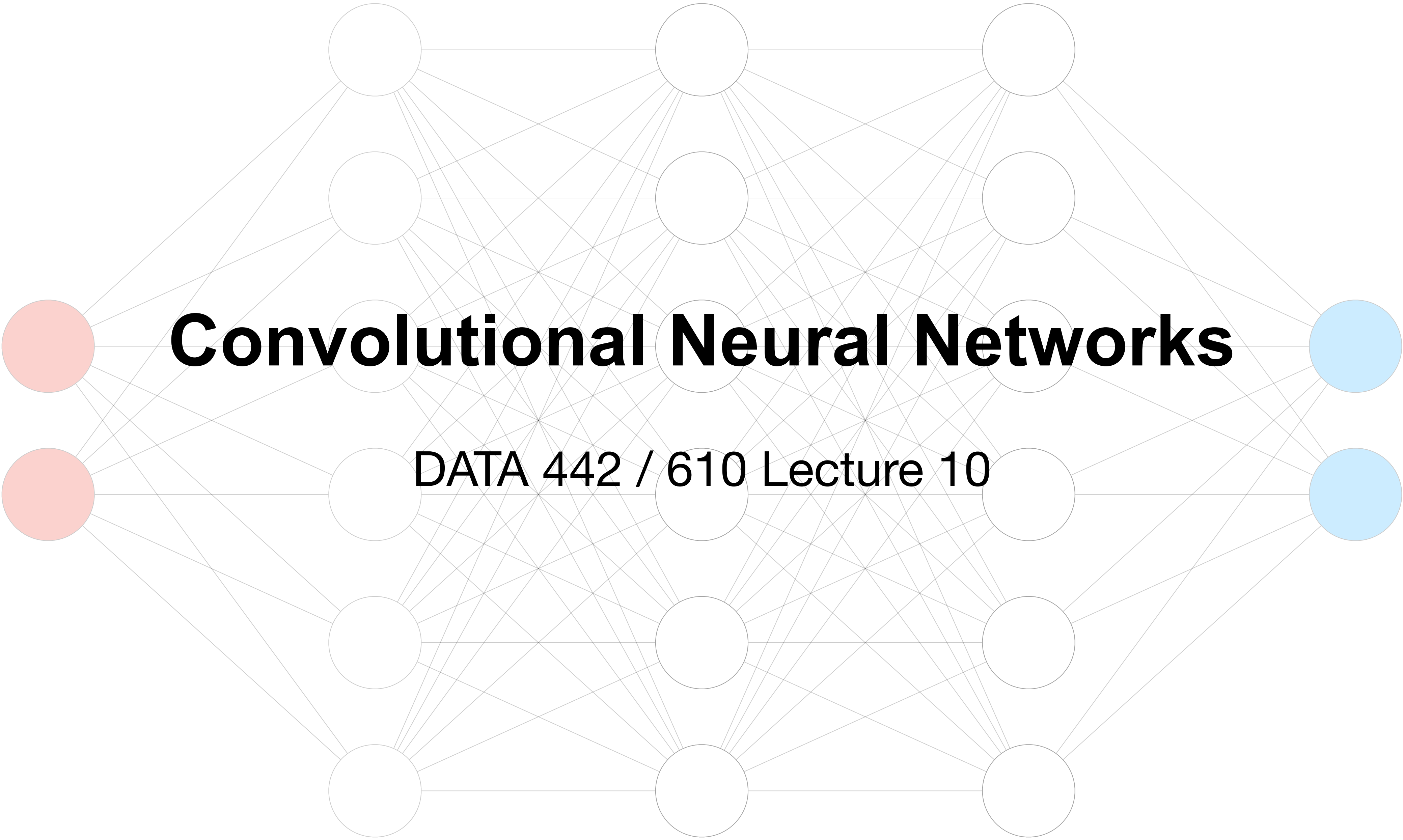




# Convolutional Neural Networks

DATA 442 / 610 Lecture 10

# Outline

- Review CNNs
- CNN demo

**Relevant Reading:** *Deep Learning*, Chapter 9



Jefferson Lab

U.S. DEPARTMENT  
of ENERGY

WDTS  
WORKFORCE DEVELOPMENT  
FOR YOUNG MEN & WOMEN

**Jan. 4-8, 2027**

**A paid, residential experience for undergrads and recent graduates to explore science, technology, engineering and computing careers within the U.S. Department of Energy national laboratory system.**

**Applications must include:**

- A 500-word essay describing interests, goals and reasons for applying
- An unofficial college transcript (or fall 2026 course schedule for first-year undergrads)
- Contact information for 1-2 recommenders

For complete information on  
**program benefits, eligibility and  
application requirements, and to apply, visit**  
**[jlab.org/stem/genesishorizons](https://jlab.org/stem/genesishorizons) or scan here:**



**Submit your application by: Wednesday, Nov. 4, at 11:59 p.m. EST**

**Questions? Email:** [GenesisHorizons@jlab.org](mailto:GenesisHorizons@jlab.org)

## Program Structure

Genesis Horizons is an intensive, five-day program that combines technical exposure, professional development, career exploration, and guided application support. Participants will attend the program from approximately 8:30 a.m. to 4:30 p.m. each day and must be available for the entire program.

Program activities are expected to include:

- Introductions to Jefferson Lab, the DOE laboratory complex, and WDTS opportunities
- Presentations and discussions related to the Genesis Mission, artificial intelligence, data science, and advanced computing
- Exposure to various departments at Jefferson Lab, including but not limited to Continuous Electron Beam Accelerator Facility (CEBAF); the Electron Ion Collider (EIC); the High Performance Data Facility (HPDF); Energy and Applied Science and Technology (EAST) - Research & Development, Engineering; Fundamental Physics; Data and Computational Science; and Environment, Safety, and Health (ES&H)
- Facility and technical tours
- Career panels, reverse interviews, reverse poster sessions, and working lunches with Jefferson Lab professionals
- Dedicated guidance and work time for completing a Summer 2027 [SULI](#) or [CCI](#) application
- A collaborative group reflection presentation at the conclusion of the program

Program activities and tours are subject to operational conditions and final availability.

## Program Benefits

Genesis Horizons participants will gain a deeper understanding of national laboratory research, DOE mission priorities, WDTS opportunities, and the wide range of STEM careers available within a national laboratory environment.

Selected participants will also receive:

- A \$350 stipend following successful completion of the program
- Shared lodging at the [SURA Residence Facility](#)
- Eligible travel support
- Working lunches throughout the program
- Direct engagement with scientists, engineers, computer scientists, technicians, and other Jefferson Lab professionals
- Structured support for preparing and submitting a Summer 2027 [SULI](#) or [CCI](#) application
- Opportunities to strengthen communication, collaboration, problem solving, and professional skills
- Connections with a cohort of undergraduate students who share an interest in STEM

# Convolution

2-D discrete convolution: 
$$z[i, j] = \sum_{k=-K}^K \sum_{\ell=-K}^K x[i - k, j - \ell] w[k, \ell]$$

- **Sparse connectivity:** kernel  $w_{K \times K}$  may be smaller than input
- **Parameter sharing:** same kernel  $w_{K \times K}$  is applied everywhere on the grid

# Convolution in images

Input image

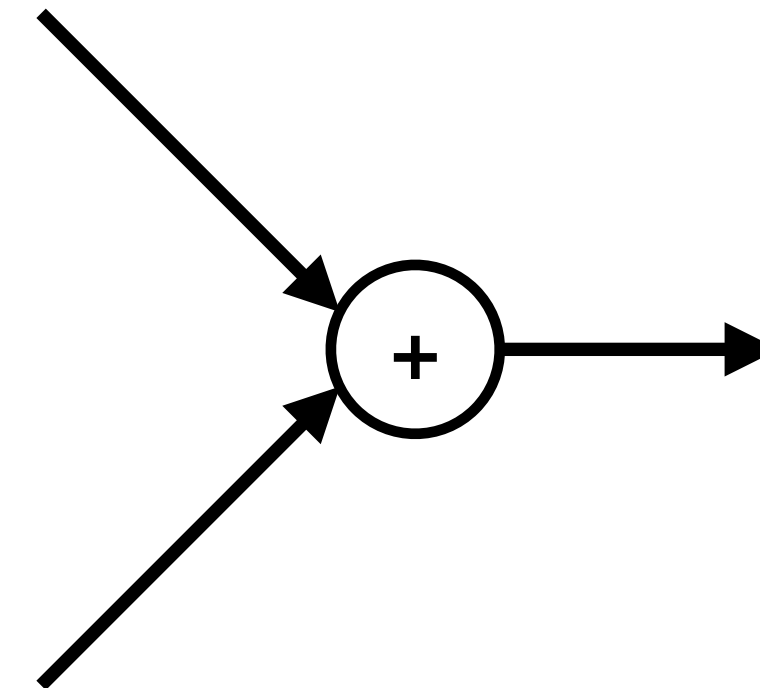
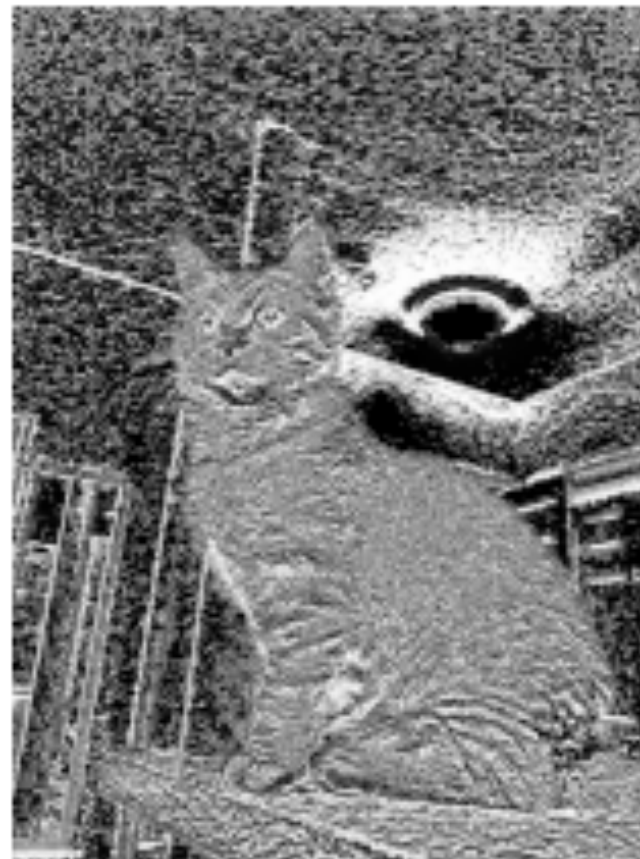
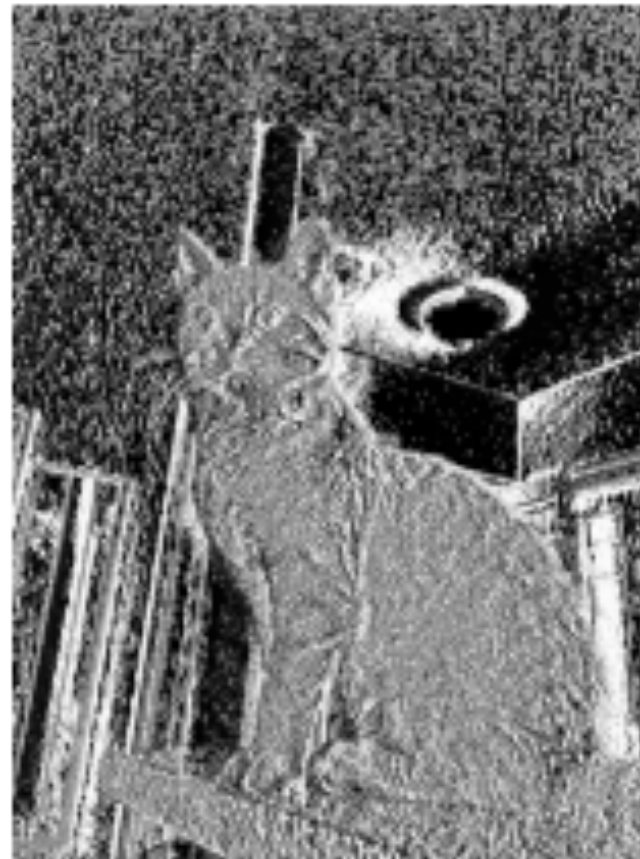


3x3 kernels

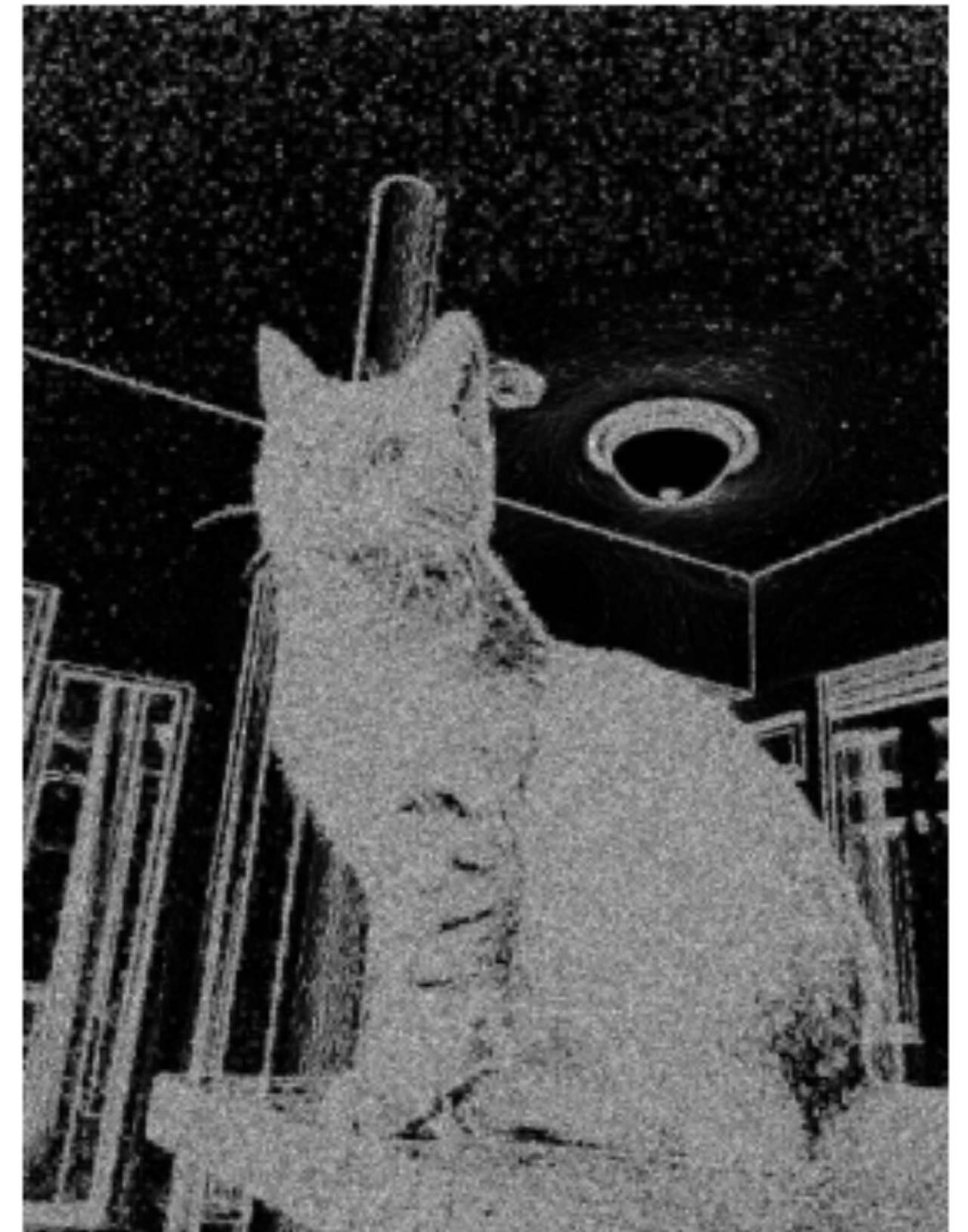
|    |   |   |
|----|---|---|
| -1 | 0 | 1 |
| -1 | 0 | 1 |
| -1 | 0 | 1 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| 0  | 0  | 0  |
| 1  | 1  | 1  |

Feature maps

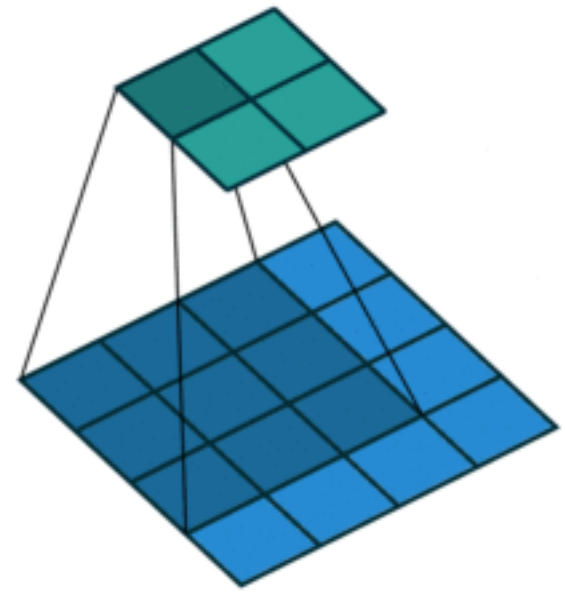


Output

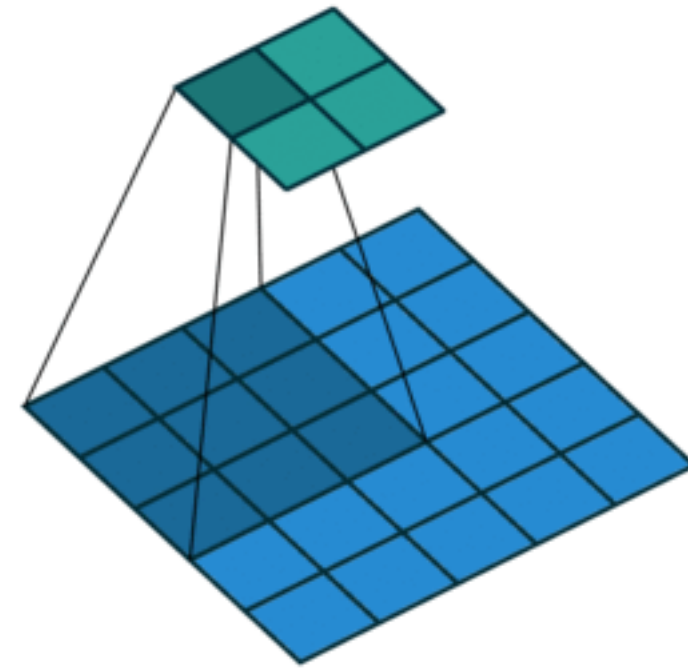


# Other useful things to know

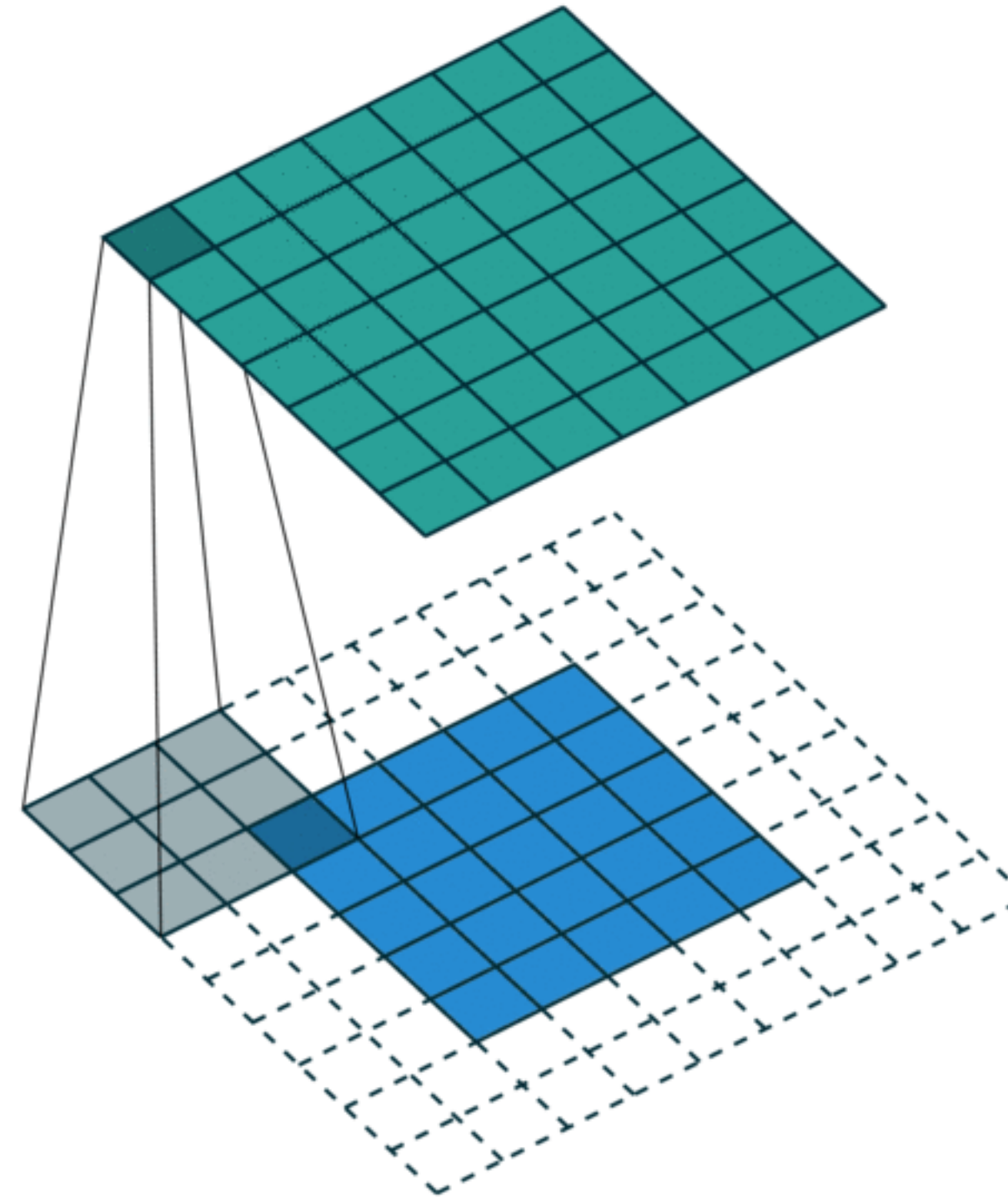
**Convolution**



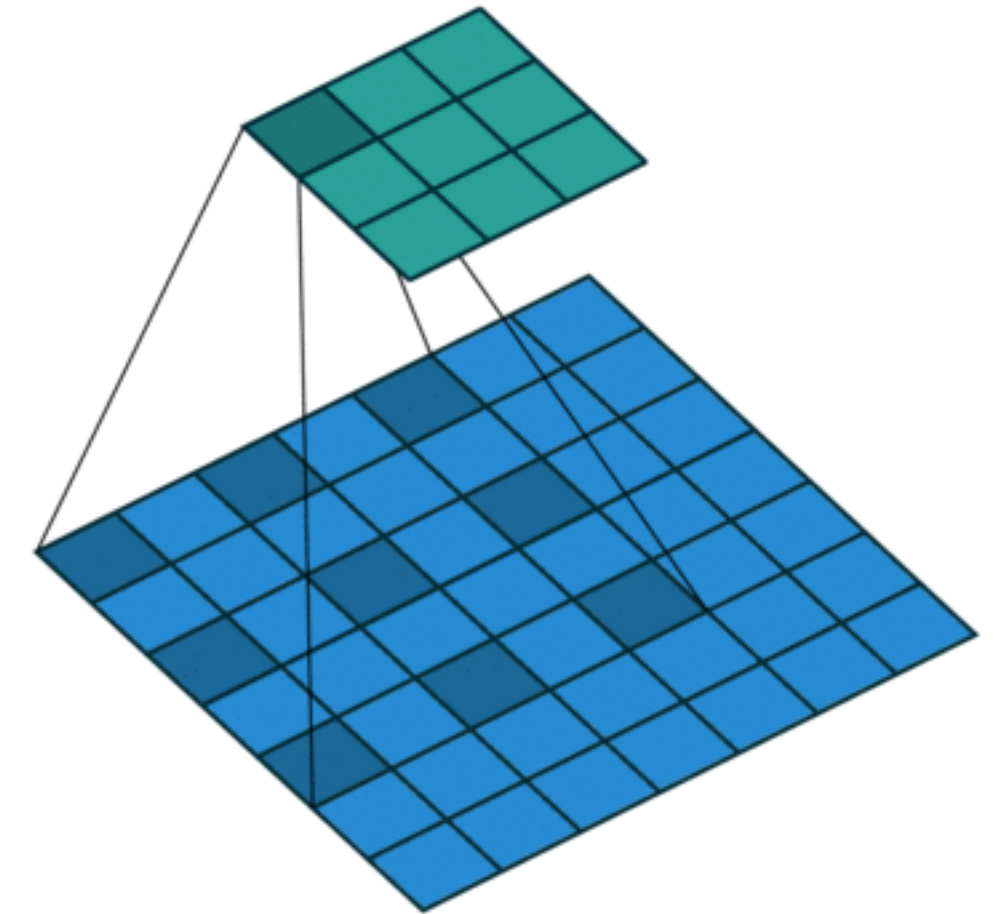
**Strided Convolution**



**Padded Convolution**



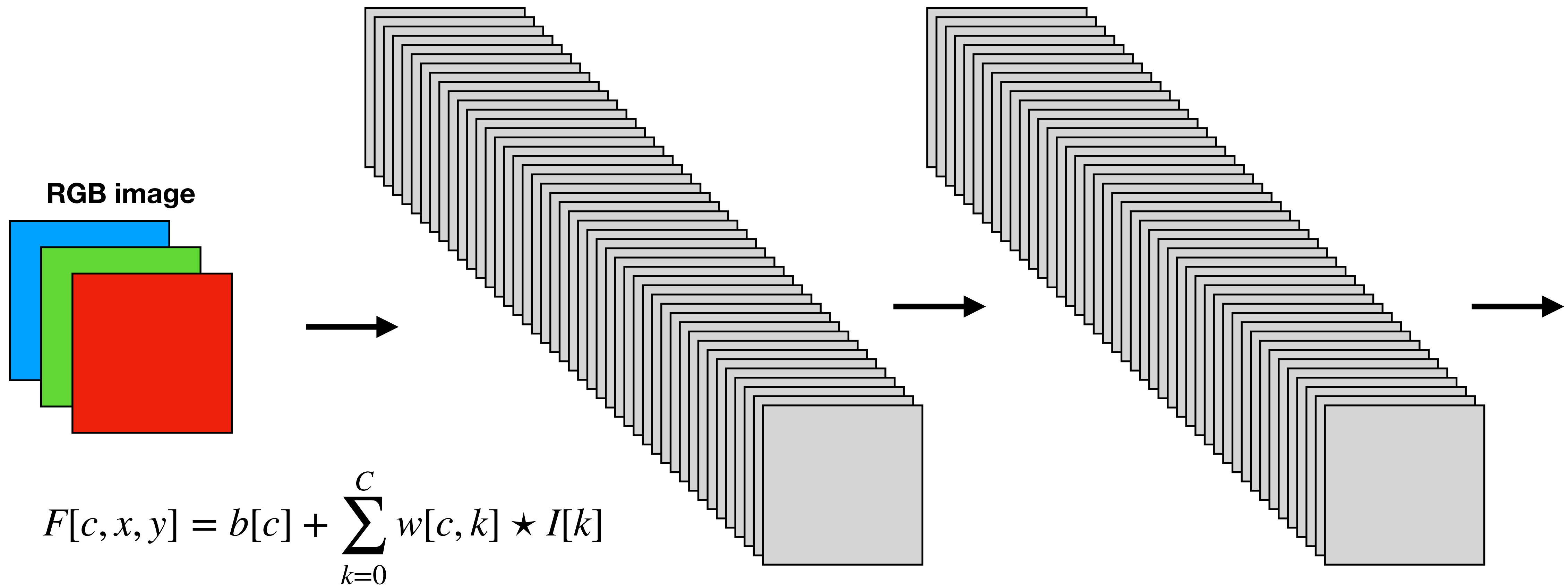
**Dilated Convolution**



See: [https://github.com/vdumoulin/conv\\_arithmetic/blob/master/README.md](https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md)

The size of an output is  $N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2 \times \text{pad} - \text{dilation} \times (\text{kernel size} - 1) - 1}{\text{stride}} + 1 \right\rfloor$

# Adding another dimension: features



Convolutional layers learn **unique kernels** between each feature map

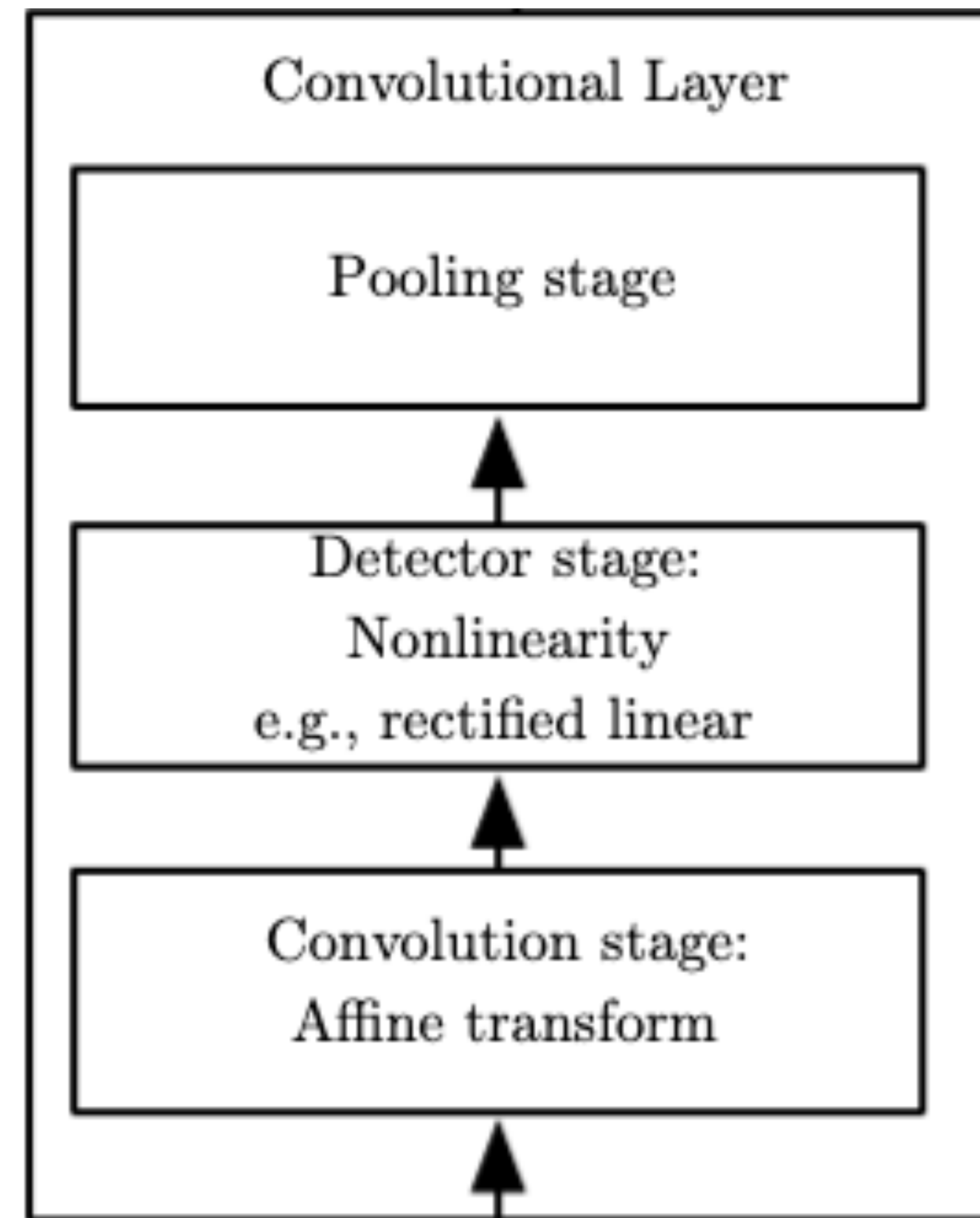
(**Parameter sharing:** a 1x1 kernel is the same as applying the same FC layer separately to each pixel)

# Convolutional Neural Networks

Pooling for **translation invariance**

Nonlinearity for **expressivity**

Sparse connectivity + shared weights for **translation equivariance**



## ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky  
University of Toronto  
kriz@cs.utoronto.ca

Ilya Sutskever  
University of Toronto  
ilya@cs.utoronto.ca

Geoffrey E. Hinton  
University of Toronto  
hinton@cs.utoronto.ca

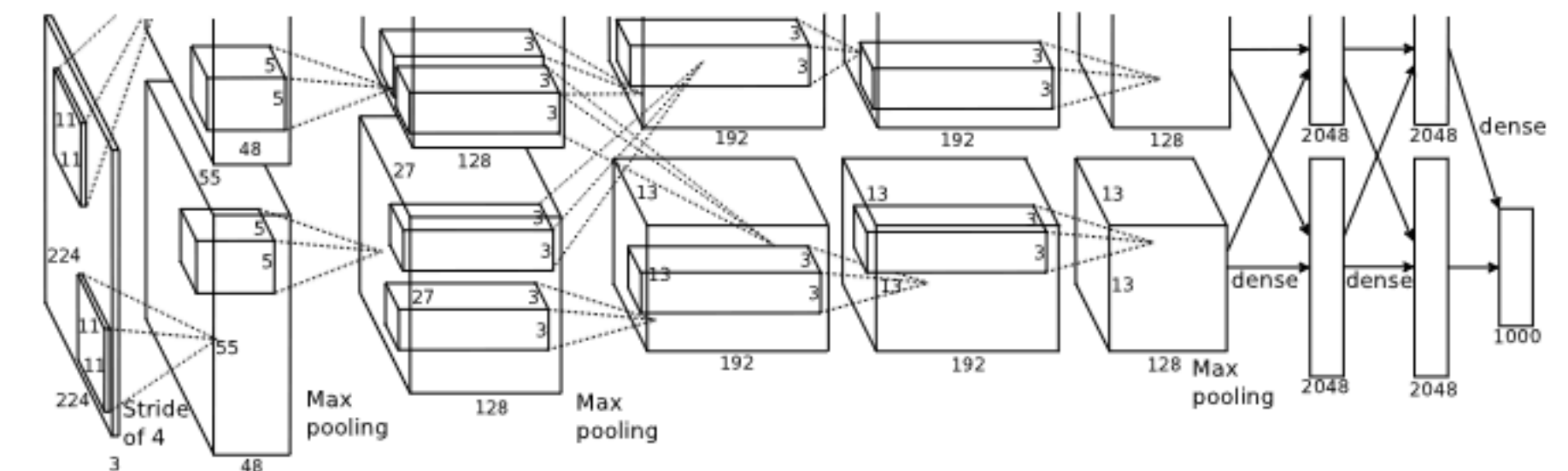


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Check out the code!

<https://github.com/computerhistory/AlexNet-Source-Code>

# Hands-on with PyTorch CNN



# Why did we talk about this?

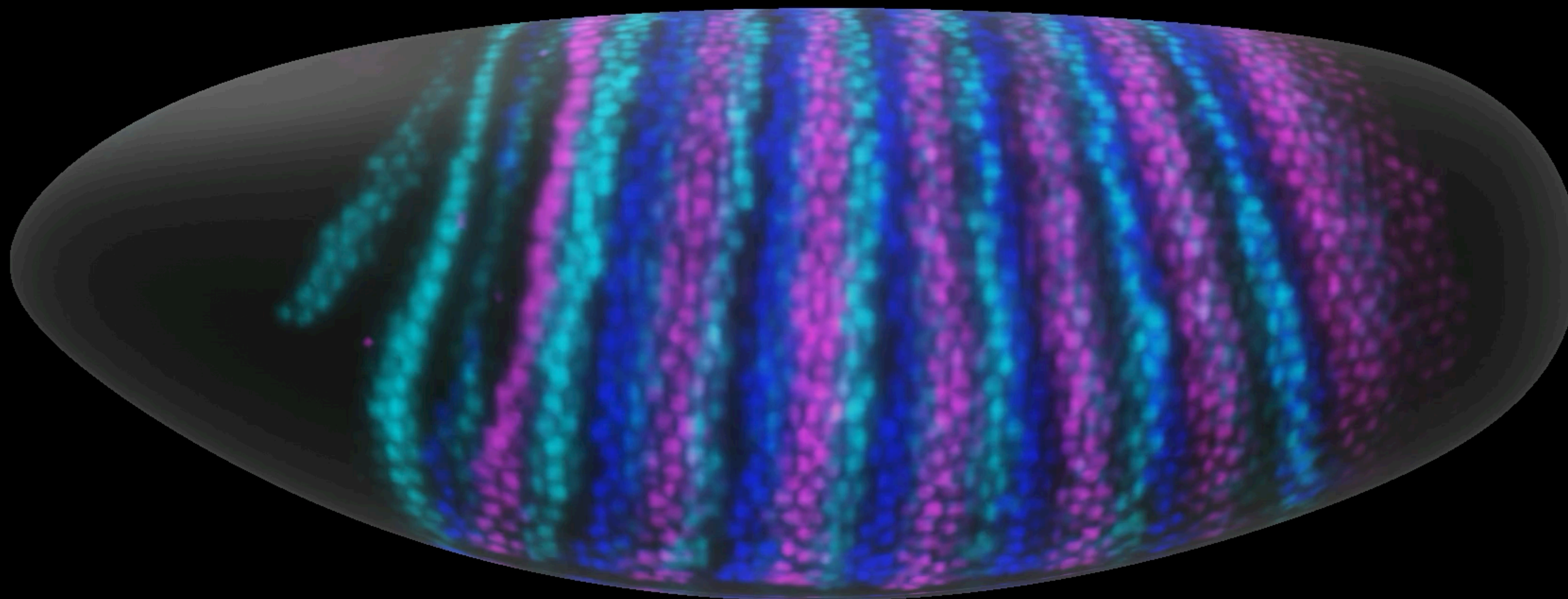
```
# Convolution that mixes channels
from numpy.lib.stride_tricks import as_strided
def multichannel_convolution_2d(x, w, pad=0, stride=1, verbose=True):
    """ X is a [N1, N2, 3] RGB image
        w is a [k1, k2, c1, c2] rank 4 tensor (a "matrix of matrices")
    """
    N1, N2, c = x.shape
    k1, k2, c_in, c_out = w.shape

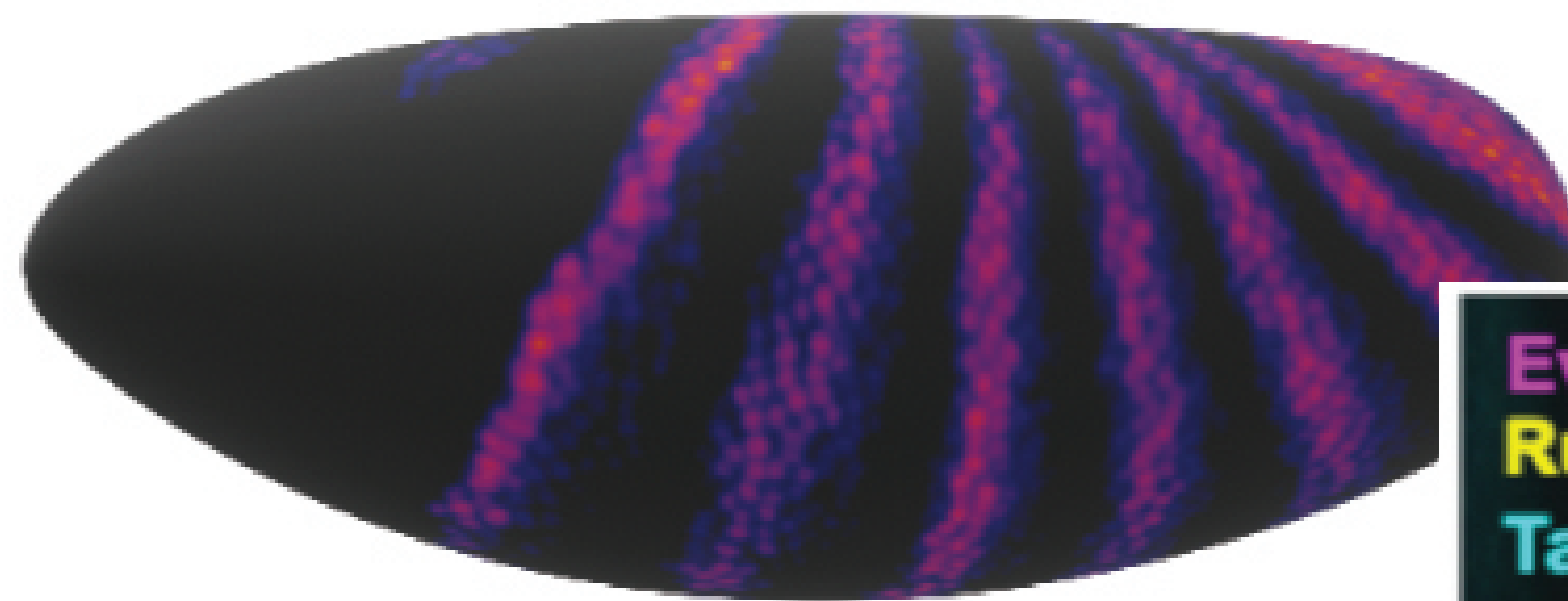
    if pad > 0:
        x_pad = np.zeros([N1+2*pad, N2+2*pad, c])
        x_pad[pad:-pad, pad:-pad] = x
    else:
        x_pad = x

    o1 = floor((N1 + 2*pad - k1) / stride + 1)
    o2 = floor((N2 + 2*pad - k2) / stride + 1)

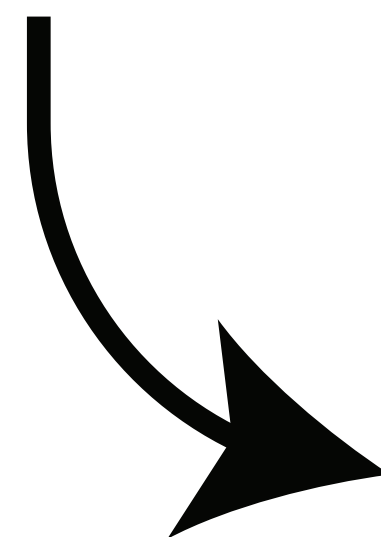
    # Instead of running nested for loops, add strides to the array to get windows equal to our kernel size
    x_stride = as_strided(
        x_pad,
        shape=(o1, # For each element in my output array along dim 0
              o2, # For each element in my output array along dim 1
              c, # For each feature at that pixel position
              k1, # Give me a matrix with k1 rows
              k2), # Give me a matrix with k2 columns
        strides=(x_pad.strides[0], # Step in dim-0 is N2 x c bytes (x is of type uint8)
                x_pad.strides[1], # Step in dim-1 is c bytes
                x_pad.strides[2], # Step in dim-2 is 1 byte (uint8)
                x_pad.strides[0], # Step in dim-3 is N2 x c bytes (move down one row)
                x_pad.strides[1]) # Step in dim-4 is c bytes (move over one column)
    )

    return np.einsum(
        'ijckl,klcd->ijd', x_stride, w[:::-1, :::-1]
    )
```

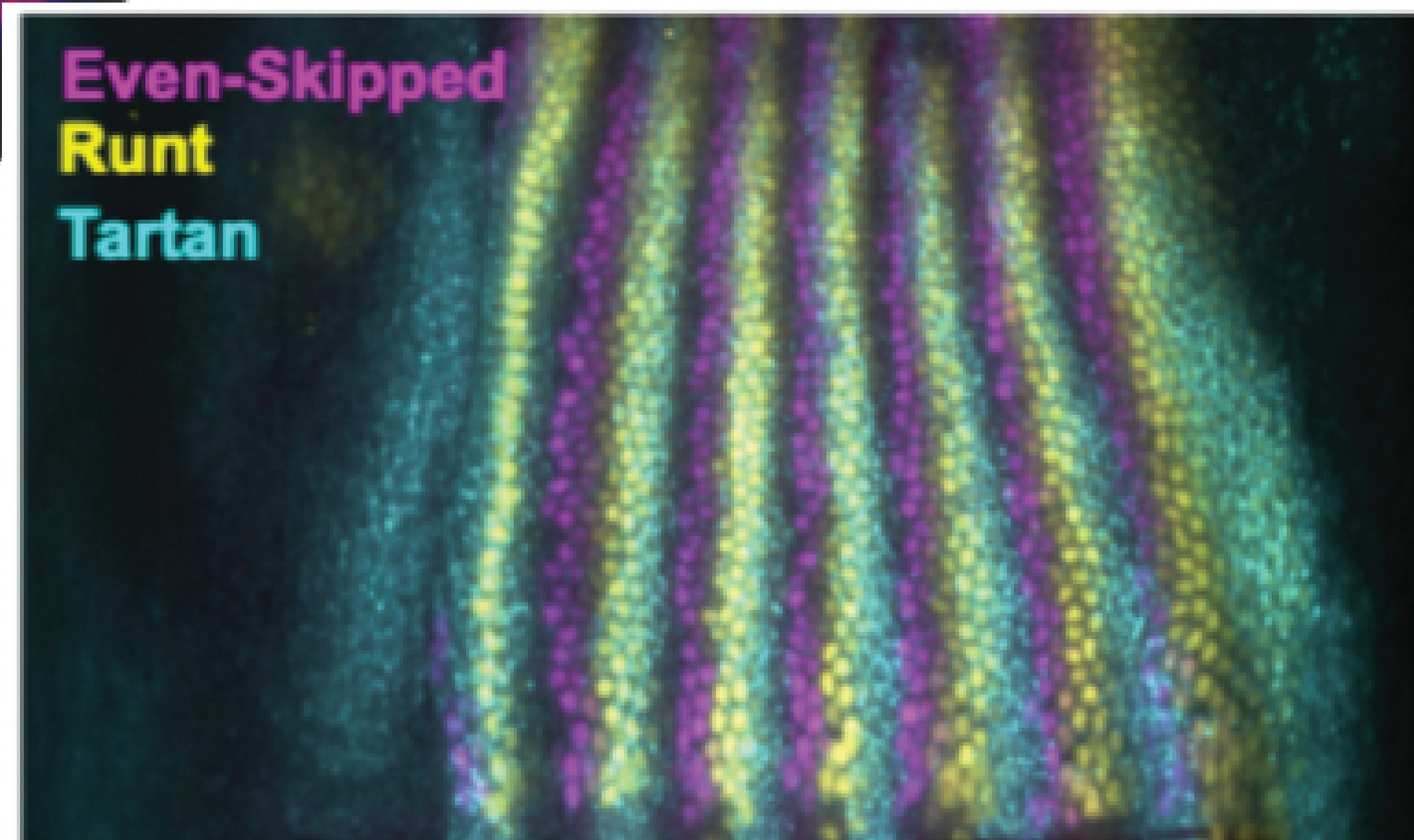




2D Cartographic  
Projection



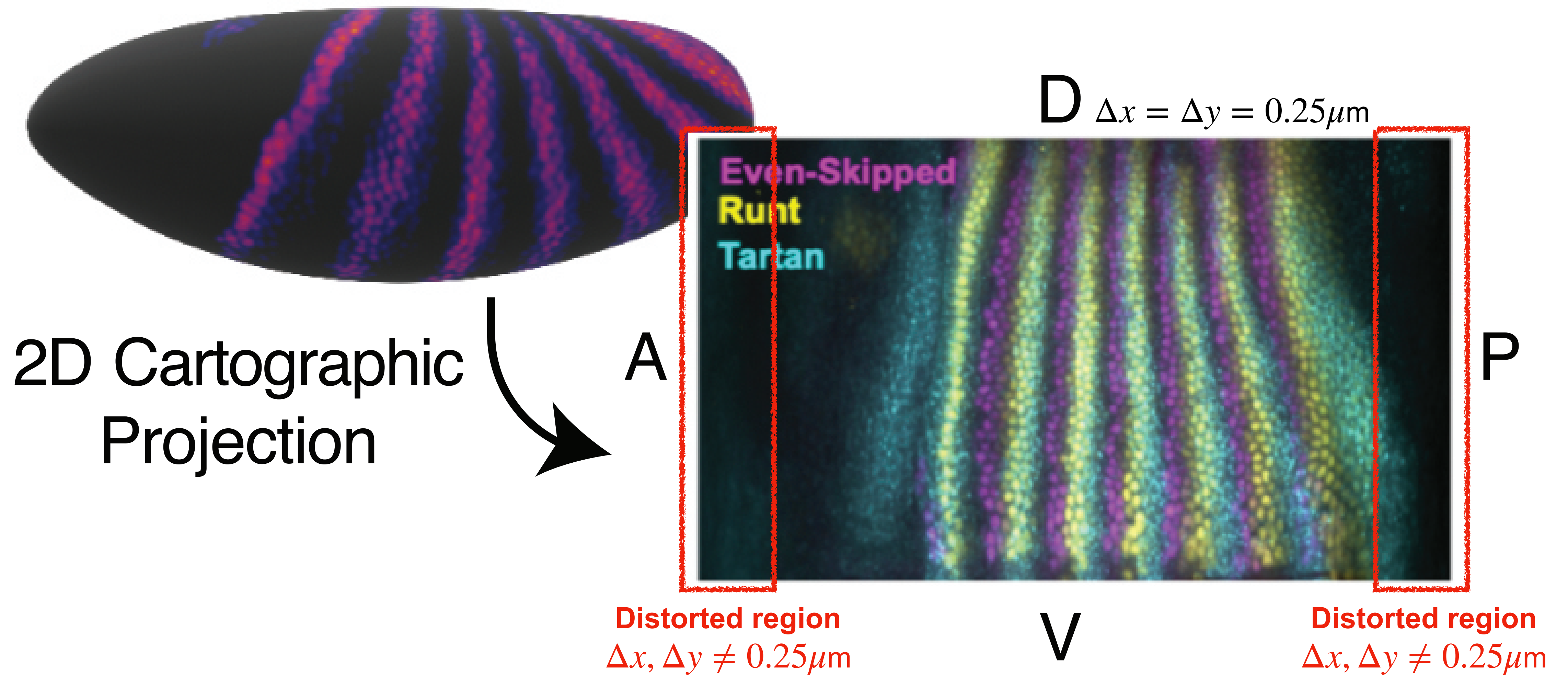
A

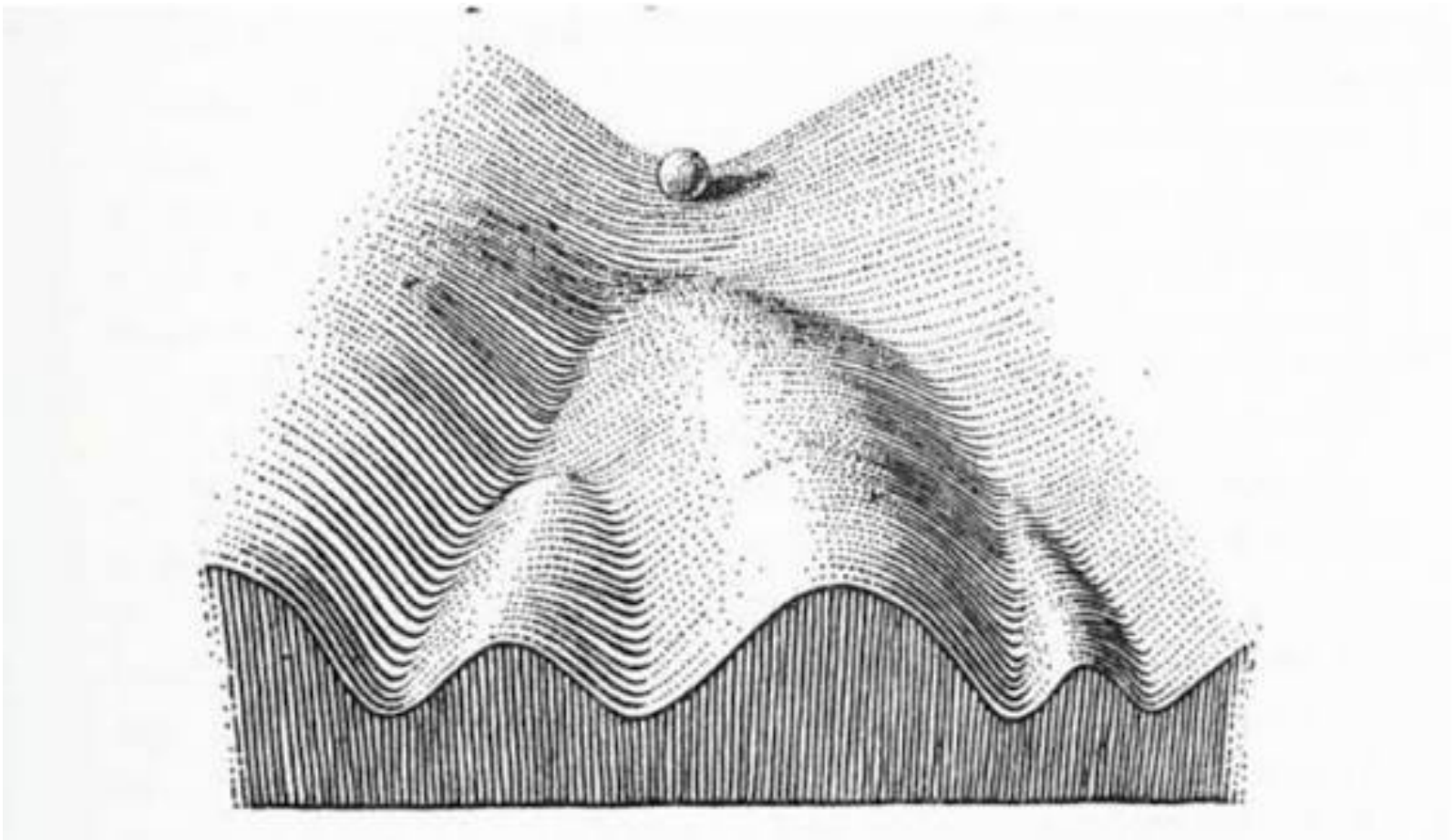
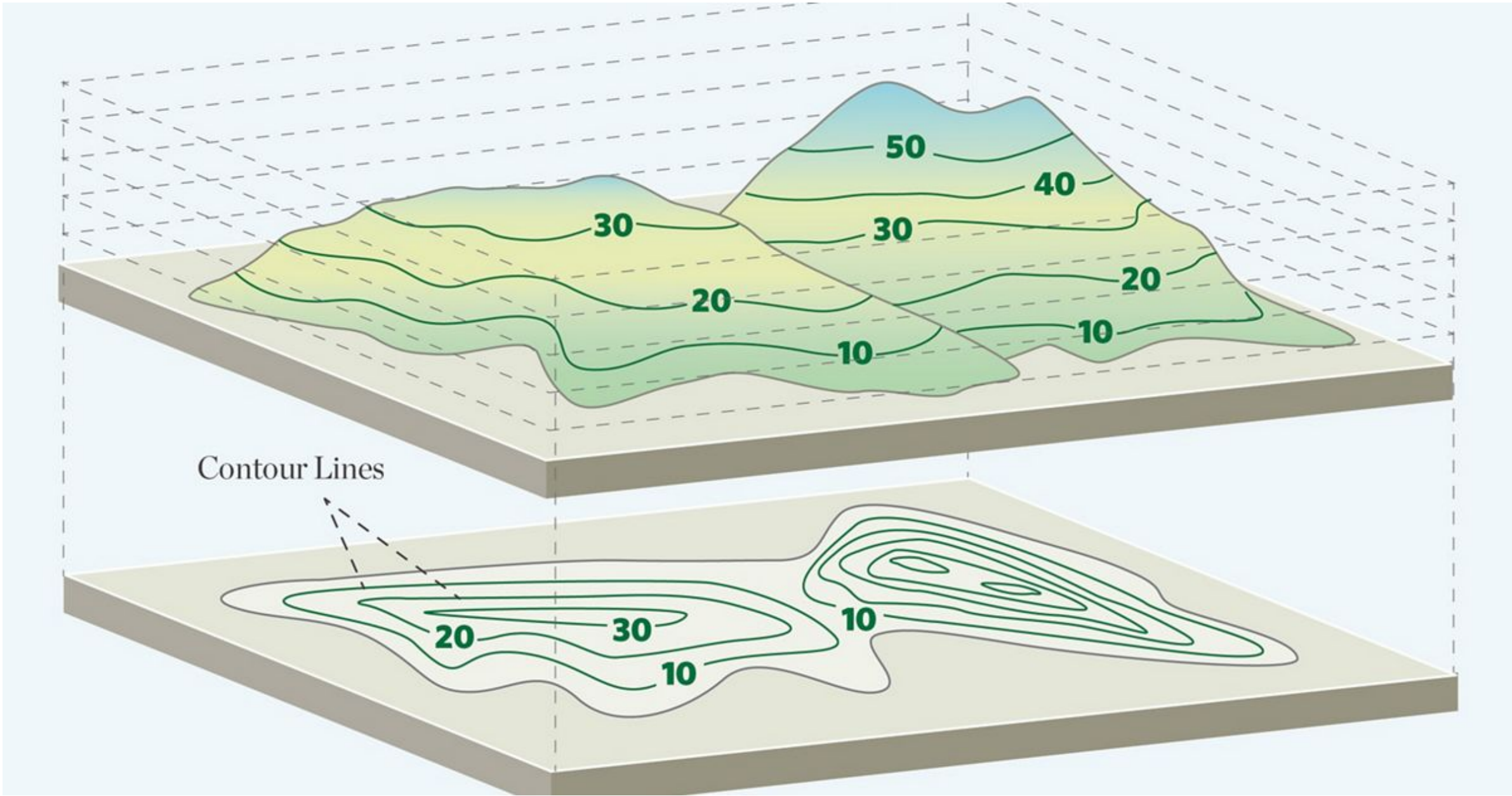


D

P

V





## Geometric deep learning on graphs and manifolds using mixture model CNNs

Federico Monti<sup>1\*</sup>

Davide Boscaini<sup>1\*</sup>

Jonathan Masci<sup>1,4</sup>

Emanuele Rodolà<sup>1</sup>

Jan Svoboda<sup>1</sup>

Michael M. Bronstein<sup>1,2,3</sup>

<sup>1</sup>USI Lugano

<sup>2</sup>Tel Aviv University

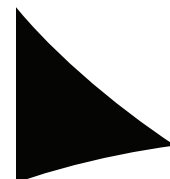
<sup>3</sup>Intel Perceptual Computing

<sup>4</sup>Nnaisense

The main contribution of this paper is a generic spatial-domain framework for deep learning on non-Euclidean domains such as graphs and manifolds. We use  $x$  to denote, depending on context, a point on a manifold or a vertex of a graph, and consider points  $y \in \mathcal{N}(x)$  in the neighborhood of  $x$ . With each such  $y$ , we associate a  $d$ -dimensional vector of *pseudo-coordinates*  $\mathbf{u}(x, y)$ . In these coordinates, we define a weighting function (kernel)  $\mathbf{w}_{\Theta}(\mathbf{u}) = (w_1(\mathbf{u}), \dots, w_J(\mathbf{u}))$ , which is parametrized by some learnable parameters  $\Theta$ . The patch operator can therefore be written in the following general form

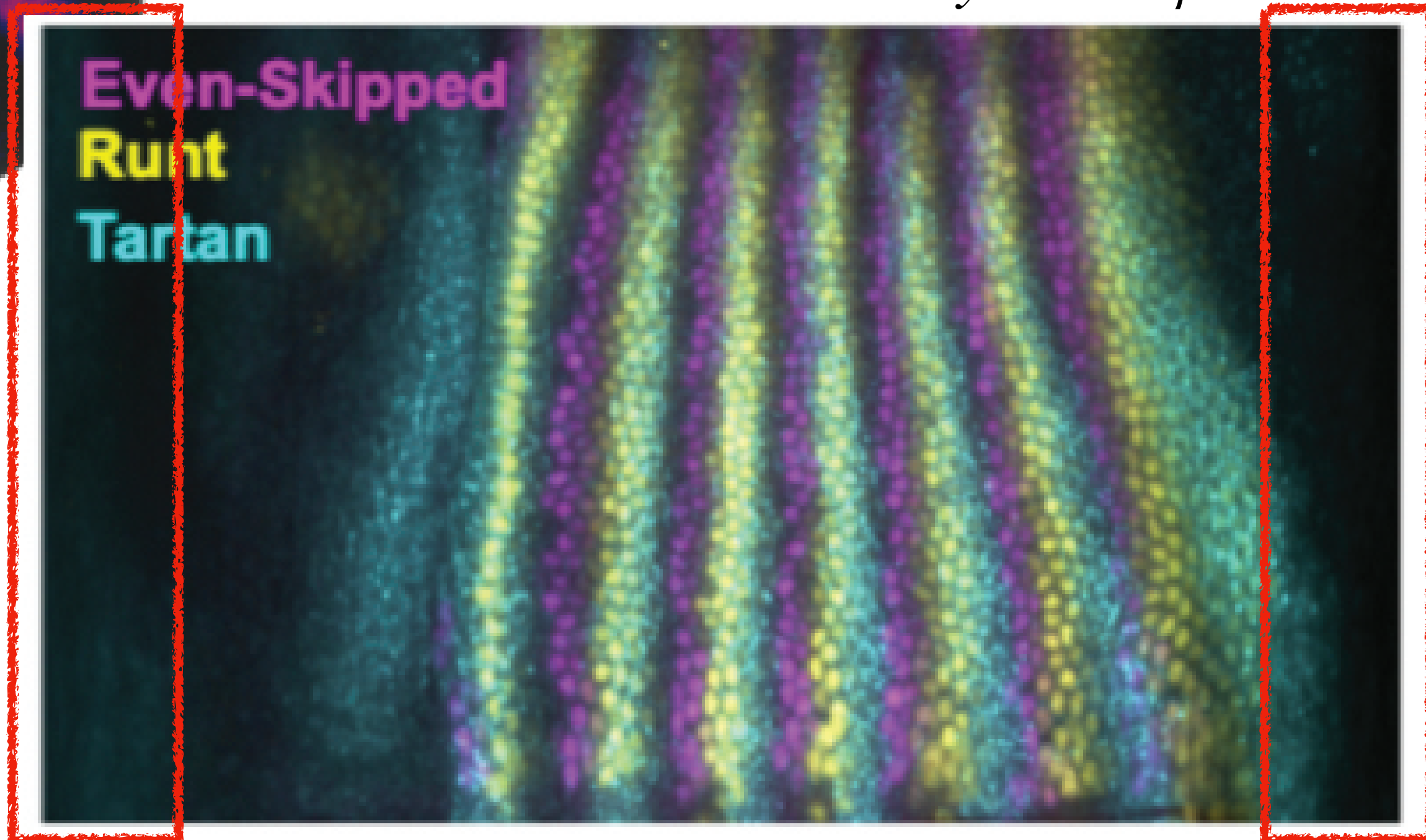
$$D_j(x)f = \sum_{y \in \mathcal{N}(x)} w_j(\mathbf{u}(x, y))f(y), \quad j = 1, \dots, J, \quad (9)$$

A



Distorted region  
 $\Delta x, \Delta y \neq 0.25\mu\text{m}$

D  $\Delta x = \Delta y = 0.25\mu\text{m}$



P

Distorted region  
 $\Delta x, \Delta y \neq 0.25\mu\text{m}$

V

## Geometric deep learning on graphs and manifolds using mixture model CNNs

Federico Monti<sup>1\*</sup>

Davide Boscaini<sup>1\*</sup>

Jonathan Masci<sup>1,4</sup>

Emanuele Rodolà<sup>1</sup>

Jan Svoboda<sup>1</sup>

Michael M. Bronstein<sup>1,2,3</sup>

<sup>1</sup>USI Lugano

<sup>2</sup>Tel Aviv University

<sup>3</sup>Intel Perceptual Computing

<sup>4</sup>Nnaisense

The main contribution of this paper is a generic spatial-domain framework for deep learning on non-Euclidean domains such as graphs and manifolds. We use  $x$  to denote, depending on context, a point on a manifold or a vertex of a graph, and consider points  $y \in \mathcal{N}(x)$  in the neighborhood of  $x$ . With each such  $y$ , we associate a  $d$ -dimensional vector of *pseudo-coordinates*  $\mathbf{u}(x, y)$ . In these coordinates, we define a weighting function (kernel)  $\mathbf{w}_{\Theta}(\mathbf{u}) = (w_1(\mathbf{u}), \dots, w_J(\mathbf{u}))$ , which is parametrized by some learnable parameters  $\Theta$ . The patch operator can therefore be written in the following general form

$$D_j(x)f = \sum_{y \in \mathcal{N}(x)} w_j(\mathbf{u}(x, y))f(y), \quad j = 1, \dots, J, \quad (9)$$

A

Distorted  
 $\Delta x, \Delta y \neq$

```
def view_as_windows(arr_in, window_shape, step=1):
    arr_shape = torch.tensor(arr_in.shape)
    window_shape = torch.tensor(window_shape)
    if len(window_shape) == 0:
        window_shape = torch.tensor([window_shape, window_shape])
    step = torch.tensor(step)
    if len(step.shape) == 0:
        step = step.view(1).expand(4)

    slices = tuple(slice(None, None, st) for st in step)
    window_strides = arr_in.stride()[2:]
    indexing_strides = arr_in[slices].stride()
    win_indices_shape = (((arr_shape[2:] - window_shape) // step[2:]) + 1)
    new_shape = tuple(list(arr_in.shape[:2]) +
                      win_indices_shape.tolist() +
                      window_shape.tolist())

    strides = tuple(list(indexing_strides) + list(window_strides))
    arr_out = arr_in.as_strided(size=new_shape, stride=strides)
    return arr_out
```

```
def forward(self, x, metric):
    b, c, h, w = x.shape
    newh = (h + 2 * self.padding[-2] - self.neighborhood) // self.stride[-2] + 1
    neww = (w + 2 * self.padding[-1] - self.neighborhood) // self.stride[-1] + 1

    #Get patches to convolve over
    x = F.pad(x, self.padding)
    neighborhoods = view_as_windows(x, (self.neighborhood, self.neighborhood), step=self.stride)
    neighborhoods = neighborhoods.reshape([b, c, newh*neww, self.neighborhood*self.neighborhood])

    #Compute weights within patch
    ex = torch.sqrt(metric[0]).flatten()
    ey = torch.sqrt(metric[-1]).flatten()
    pseudo = torch.stack((
        torch.einsum('a,b', ex, self.dists[:, 0]),
        torch.einsum('a,b', ey, self.dists[:, 1])), dim=-1)

    weights = (-0.5 * torch.sum((pseudo[..., None, :] - self.means)**2 * self.invcovs, axis=-1)).exp()
    strided_mat = torch.sum(neighborhoods[..., None] * weights[None, None], dim=-2)

    #Apply conv kernel over patches
    strided_kernel = self.weight.flatten(start_dim=-2)
    conv = torch.einsum('ijk,ajbk->aib', strided_kernel, strided_mat) + self.bias[None, :, None]
    conv = conv.reshape([b, self.out_channels, h, w])

    return conv
```

P

region  
 $0.25\mu\text{m}$

[Submitted on 27 Apr 2021 (v1), last revised 2 May 2021 (this version, v2)]

# Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges

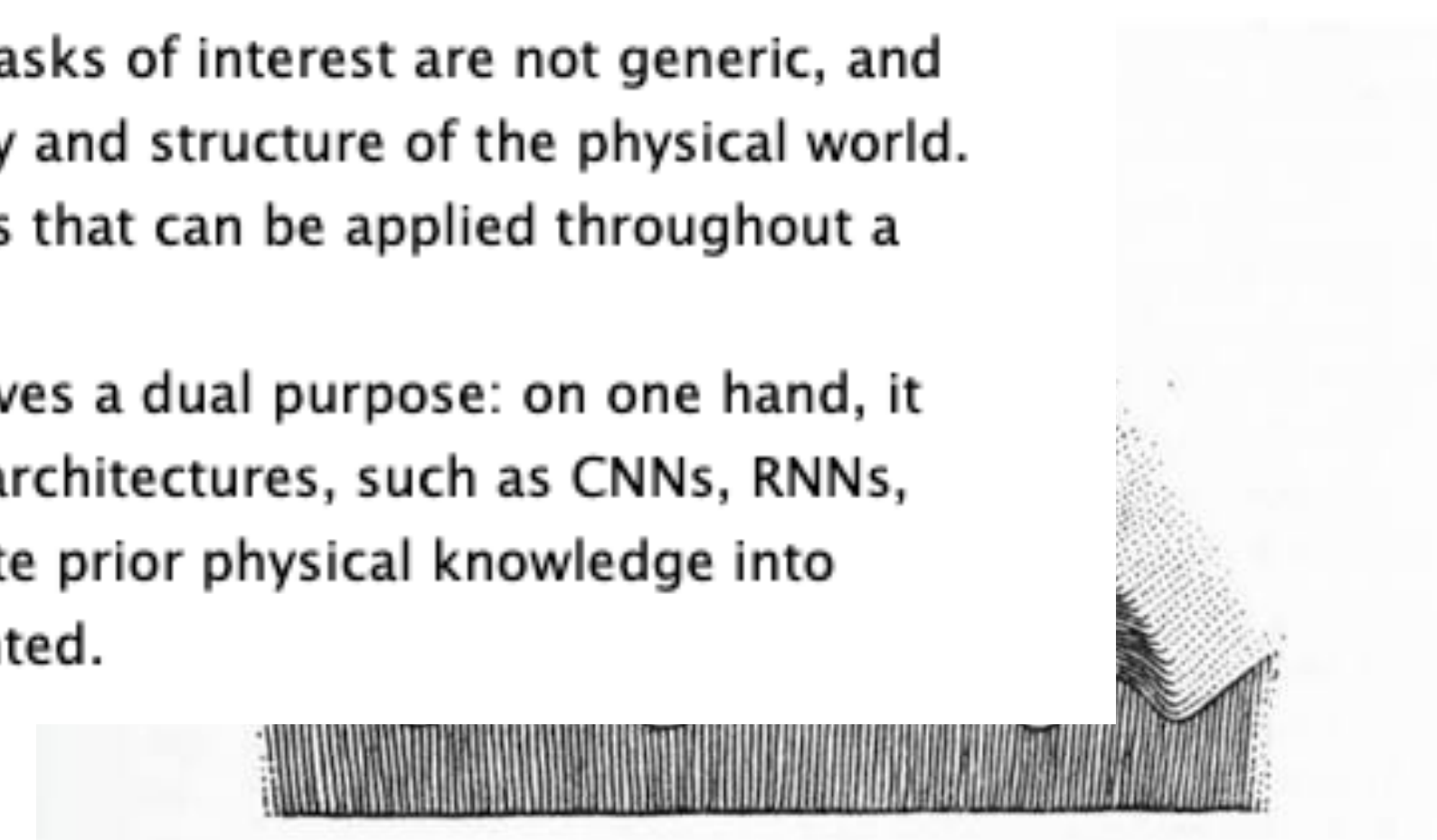
Michael M. Bronstein, Joan Bruna, Taco Cohen, Petar Veličković

The last decade has witnessed an experimental revolution in data science and machine learning, epitomised by deep learning methods. Indeed, many high-dimensional learning tasks previously thought to be beyond reach -- such as computer vision, playing Go, or protein folding -- are in fact feasible with appropriate computational scale. Remarkably, the essence of deep learning is built from two simple algorithmic principles: first, the notion of representation or feature learning, whereby adapted, often hierarchical, features capture the appropriate notion of regularity for each task, and second, learning by local gradient-descent type methods, typically implemented as backpropagation.

While learning generic functions in high dimensions is a cursed estimation problem, most tasks of interest are not generic, and come with essential pre-defined regularities arising from the underlying low-dimensionality and structure of the physical world. This text is concerned with exposing these regularities through unified geometric principles that can be applied throughout a wide spectrum of applications.

Such a 'geometric unification' endeavour, in the spirit of Felix Klein's Erlangen Program, serves a dual purpose: on one hand, it provides a common mathematical framework to study the most successful neural network architectures, such as CNNs, RNNs, GNNs, and Transformers. On the other hand, it gives a constructive procedure to incorporate prior physical knowledge into neural architectures and provide principled way to build future architectures yet to be invented.

<https://arxiv.org/pdf/2104.13478>



# Hands-on with PyTorch CNN



# That's all!

- Slides will be posted on the course website: <https://colen-teaching.github.io/DATA442-Fall-2026>
- Lab demos will also be posted
- **PS2** due October 23